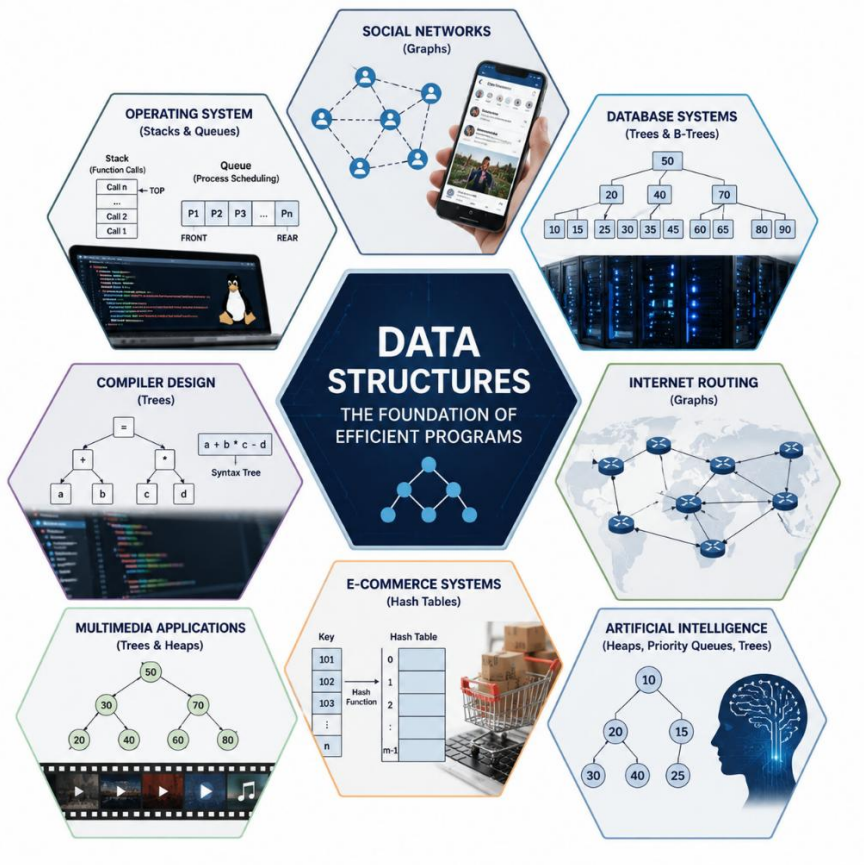
Data Structures & Algorithms Lab

ECE2132



Department of Electronics & Communication Engineering

By:

Dr. D.V. Prashant
Dr. Kamal Kishore Upadhyay

Manipal University Jaipur,

Dehmi Kalan, Off Jaipur-Ajmer Expressway, Jaipur, (Raj.)

Rajasthan 303007

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

(AY 2026-27)

EXPERIMENT NO. 5

WRITE A PROGRAM TO IMPLEMENT MATRIX MULTIPLICATION USING MODULAR PROGRAMMING WITH SEPARATE FUNCTIONS

AIM: To study and implement matrix multiplication using modular programming in C++ by using separate functions for input, multiplication, and display of matrices

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: A matrix is a rectangular arrangement of elements organized in rows and columns. The order of a matrix is represented as m × n, where m is the number of rows and n is the number of columns.

Two matrices can be multiplied only when the number of columns of the first matrix is equal to the number of rows of the second matrix.

If matrix A is of order m × n and matrix B is of order n × p, then the resultant matrix C = A × B will be of order m × p.

Each element of the resultant matrix is obtained by multiplying the corresponding elements of a row of the first matrix with the corresponding elements of a column of the second matrix and then adding the products

For example, consider:

A = [1 2] B = [5 6]

[3 4] [7 8]

Then,

C = A × B

C = [(1×5)+(2×7) (1×6)+(2×8)]

[(3×5)+(4×7) (3×6)+(4×8)]

= [19 22]

[43 50]

Modular programming is a programming technique in which a large program is divided into smaller functions or modules. Each function performs a specific task. This makes the program easier to understand, debug, modify, and maintain.

In this experiment, separate functions are used for:

- inputMatrix() – to enter matrix elements.
- multiplyMatrix() – to perform matrix multiplication.
- displayMatrix() – to display the resultant matrix.
- main() – to control the overall execution of the program.

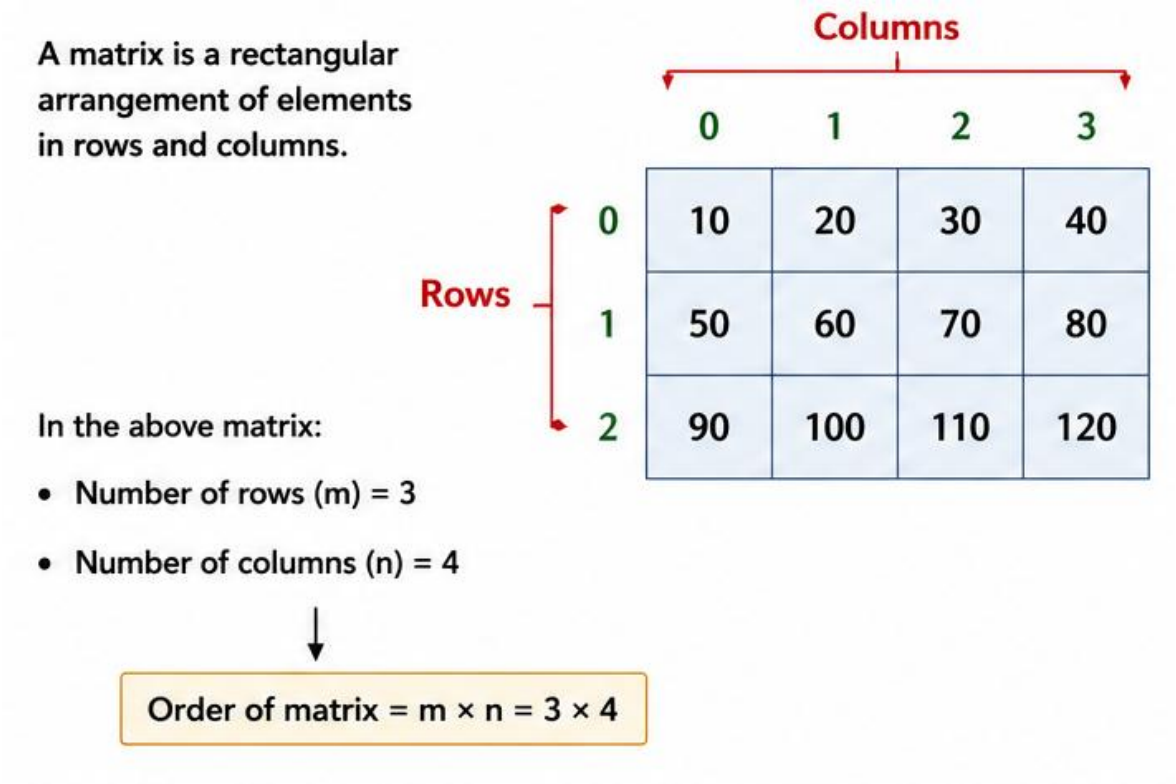


Figure. 1: Matrix Representation (Rows and Columns)

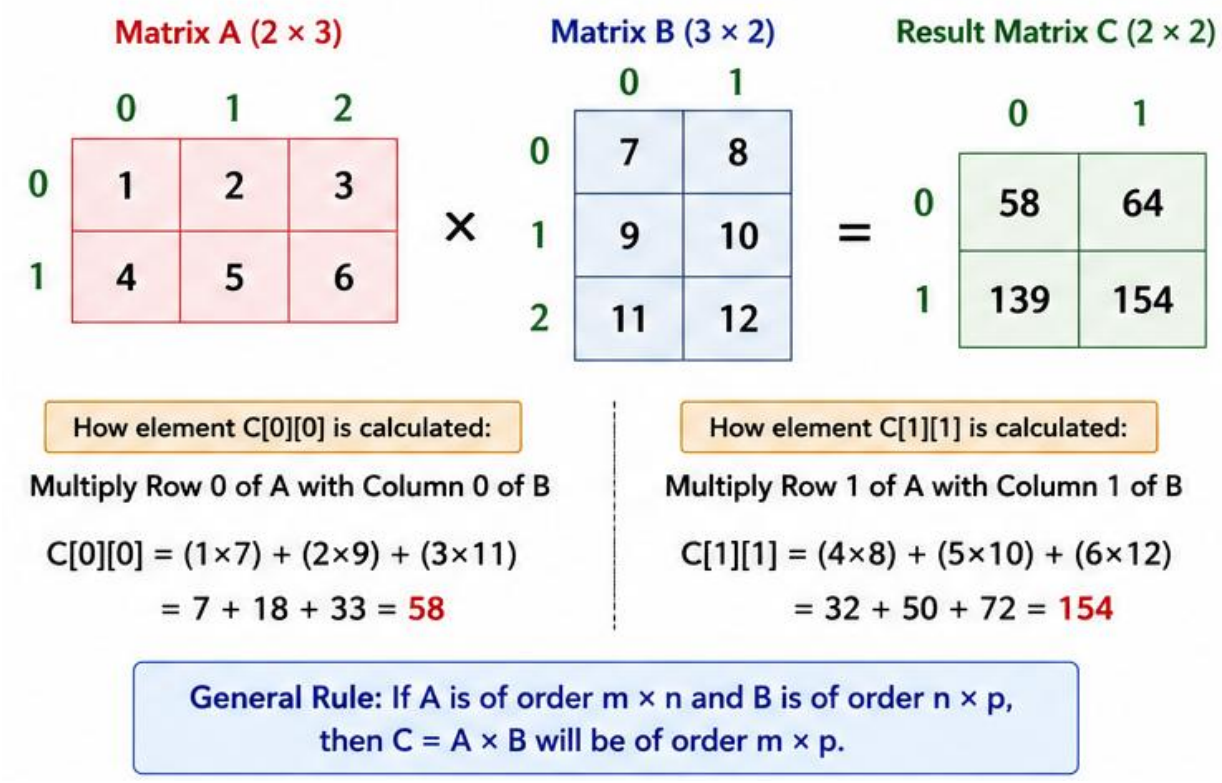


Figure. 2: Row × Column Matrix Multiplication.

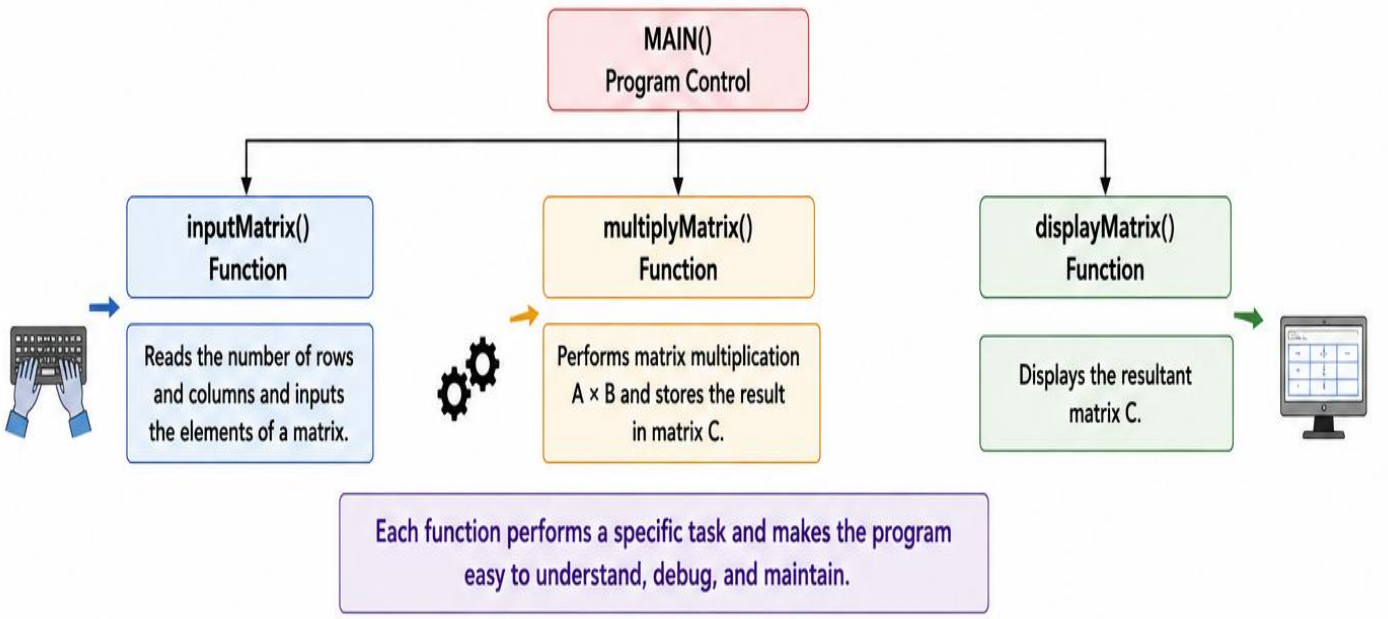


Figure. 3: Modular Programming Structure (Functions)

ALOGRITHM:

1. Start.
2. Declare matrices A, B, and C.
3. Enter the number of rows and columns of the first matrix
4. Enter the number of rows and columns of the second matrix
5. Check whether the number of columns of the first matrix is equal to the number of rows of the second matrix
6. If the condition is false, display **"Matrix multiplication not possible"**
7. If any position is selected, shift elements after the specified position towards the left.
8. If the condition is true, call inputMatrix() to enter the elements of both matrices
9. Call multiplyMatrix() to multiply the matrices
10. Store the result in matrix C.Stop.
11. Call displayMatrix() to display the resultant matrix.
12. Stop

Figure.1: Deletion of an Element from an Array

PROGRAM (C++):

```
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            cin >> matrix[i][j];
        }
    }
}

// Function to multiply two matrices
void multiplyMatrix(int A[][10], int B[][10],
                    int C[][10], int  r1, int c1, int c2)
{
    for(int i = 0; i < r1; i++)
    {
        for(int j = 0; j < c2; j++)
        {
            C[i][j] = 0;

            for(int k = 0; k < c1; k++)
            {
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
            }
        }
    }
}

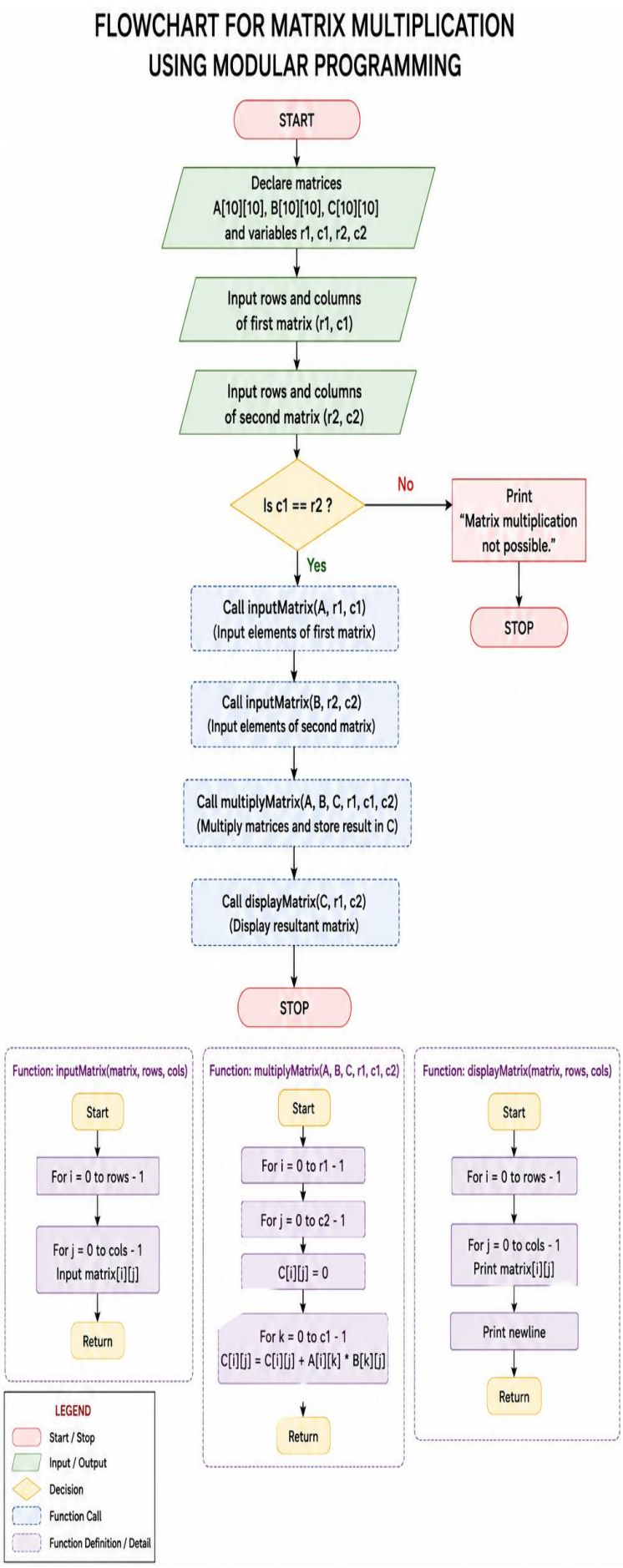
// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
}
int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    // Check multiplication condition
    if(c1 != r2)
    {
        cout << "Matrix multiplication not possible.";
        return 0;
    }
    cout << "Enter elements of first matrix:\n";
    inputMatrix(A, r1, c1);

    cout << "Enter elements of second matrix:\n";
    inputMatrix(B, r2, c2);
    // Multiply matrices
    multiplyMatrix(A, B, C, r1, c1, c2);
    cout << "\nResultant Matrix:\n";
    // Display resultant matrix
    displayMatrix(C, r1, c2);
    return 0;
}
```



EXPLANATION OF THE PROGRAM:

Statement	Explanation
int A[10][10]	Declares the first matrix.
int B[10][10]	Declares the second matrix.
int C[10][10]	Stores the resultant matrix.
r1, c1	Store rows and columns of the first matrix.
r2, c2	Store rows and columns of the second matrix
inputMatrix()	Checks whether matrix multiplication is possible
multiplyMatrix()	Performs matrix multiplication
C[i][j] = 0	Multiplies an element of a row of A with an element of a column of B.
A[i][k] * B[k][j]	Displays the updated array.
displayMatrix()	Displays the resultant matrix.
return 0;	Terminates the program successfully.

Sample Input 1 (Beginning):

Let's use simple **2 × 2 matrices:**

Enter rows and columns of first matrix:

2 2

Enter rows and columns of second matrix:

2 2

Enter elements of first matrix:

1 2

3 4

Enter elements of second matrix:

5 6

7 8.

Calculation:

First element:

C[0][0]

= 1×5 + 2×7

= 5 + 14

= 19

Second element:

C[0][1]

= 1×6 + 2×8

= 6 + 16

= 22

Third element:

C[1][0]

= 3×5 + 4×7

= 15 + 28

= 43

Fourth element:

C[1][1]

= 3×6 + 4×8

= 18 + 32

= 50

Sample Output:

Resultant Matrix:

19 22

43 50

Time Complexity

For matrices of general dimensions:

O(r1 × c1 × c2)

For two square matrices of size n × n:

$O(n^3)$

This is because the multiplication uses **three nested loops**

Space Complexity

Considering the resultant matrix:

O(r1 × c2)

For an n × n resultant matrix:

$O(n^2)$

IMPLEMENTATION OF CODE

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            cin >> matrix[i][j];
        }
    }
}

// Function to multiply two matrices
void multiplyMatrix(int A[][10], int B[][10],
                    int C[][10], int r1, int c1, int c2)
{
    for(int i = 0; i < r1; i++)
    {
        for(int j = 0; j < c2; j++)
        {
            C[i][j] = 0;

            for(int k = 0; k < c1; k++)
            {
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
            }
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            cout << matrix[i][j] << " ";
        }

        cout << endl;
    }
}

int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    // Check multiplication condition
    if(c1 != r2)
    {
        cout << "Matrix multiplication not possible.";
        return 0;
    }

    cout << "Enter elements of first matrix:\n";
    inputMatrix(A, r1, c1);

    cout << "Enter elements of second matrix:\n";
    inputMatrix(B, r2, c2);

    // Multiply matrices
    multiplyMatrix(A, B, C, r1, c1, c2);

    cout << "\nResultant Matrix:\n";

    // Display resultant matrix
    displayMatrix(C, r1, c2);

    return 0;
}
```

```
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements of first matrix:
1 2 3 4
Enter elements of second matrix:
5 6 7 8

Resultant Matrix:
19 22
43 50


...Program finished with exit code 0
Press ENTER to exit console.
```

APPLICATIONS: Computer Graphics, Image Processing, Machine Learning, Artificial Intelligence, Scientific Computing, Engineering Calculations, Robotics, Computer Vision, Transformation and Rotation Operations.

RESULT: The program was successfully executed. It demonstrated **matrix multiplication using modular programming** by using separate functions for matrix input, matrix multiplication, and displaying the resultant matrix.

ASSIGNMENT:

- WAP to add two matrices using separate functions.
- WAP to subtract two matrices using modular programming.
- WAP to find the transpose of a matrix using a separate function.
- WAP to multiply two 3 × 3 matrices using functions.
- WAP to check whether two matrices can be multiplied and display an appropriate message.

EXPERIMENT NO. 6

WRITE A PROGRAM TO IMPLEMENT MATRIX TRANSPOSE USING MODULAR PROGRAMMING WITH SEPARATE FUNCTIONS

AIM: To study and implement matrix transpose using modular programming in C++ by using separate functions for input, transpose, and display of matrices.

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: A matrix is a rectangular arrangement of elements organized in **rows and columns**. The order of a matrix is represented as **m × n**, where m is the number of rows and n is the number of columns. The reference manual introduces matrices in terms of their row and column representation and uses modular programming to divide operations into separate functions.

Matrix Transpose

The **transpose of a matrix** is obtained by interchanging its rows and columns.

If matrix A is of order m × n, then its transpose T will be of order n × m.

The element at position A[i][j] is placed at position T[j][i].

Therefore,

T[j][i] = A[i][j]

Example

Consider the matrix:

A = [1 2 3]
[4 5 6]

The order of matrix A is **2 × 3**.

After taking transpose:

T = [1 4]
[2 5]
[3 6]

The order of the transpose is **3 × 2**.

The elements are transferred as follows:

A[0][0] → T[0][0]
A[0][1] → T[1][0]
A[0][2] → T[2][0]

A[1][0] → T[0][1]
A[1][1] → T[1][1]
A[1][2] → T[2][1]

Modular Programming

Modular programming is a programming technique in which a large program is divided into smaller functions or modules. Each function performs a specific task. This makes the program easier to understand, debug, modify, and maintain. This follows the modular-programming approach used in the reference experiment.

In this experiment, separate functions are used for:

- inputMatrix() – to enter matrix elements.
- transposeMatrix() – to calculate the transpose of the matrix.
- displayMatrix() – to display the matrix.
- main() – to control the overall execution of the program.

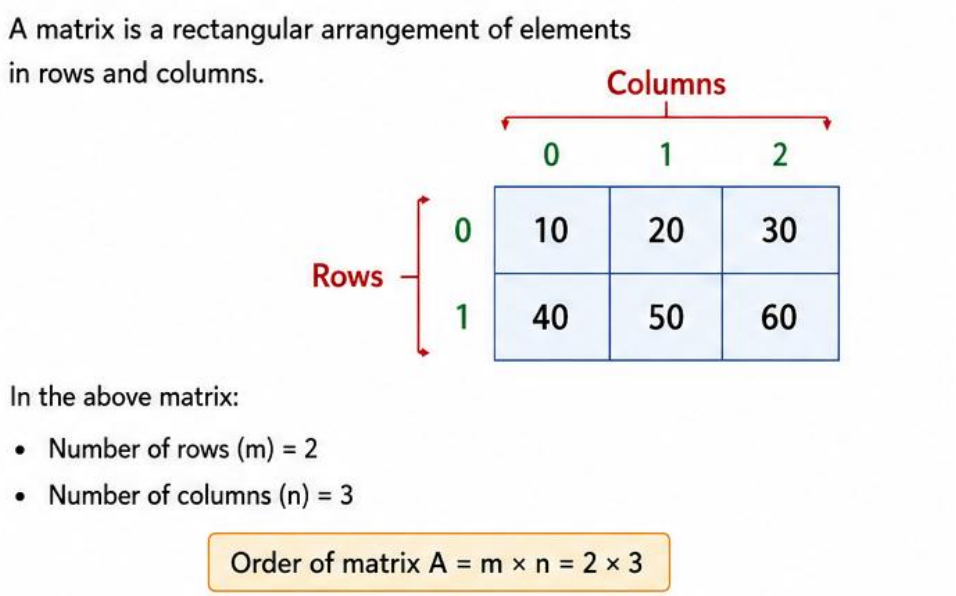


Figure. 1: Matrix Representation (Rows and Columns)

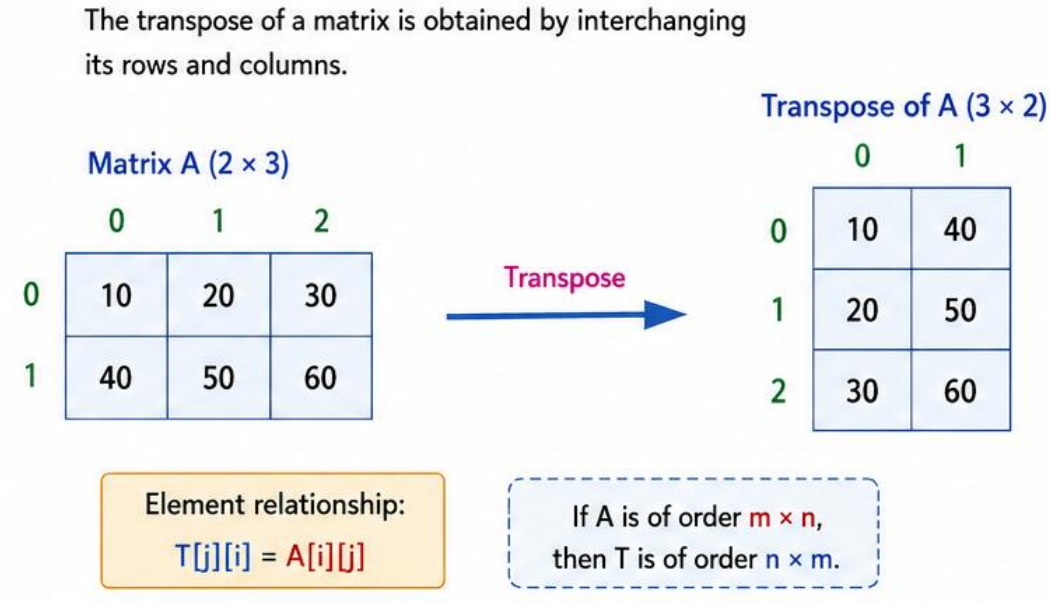


Figure. 2: Matrix Transpose.

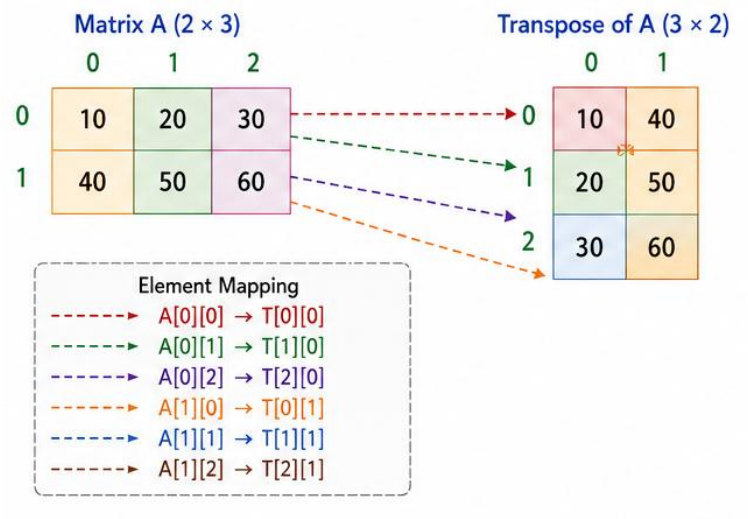


Figure. 3: Element Mapping (A[i][j] to T[i][j])

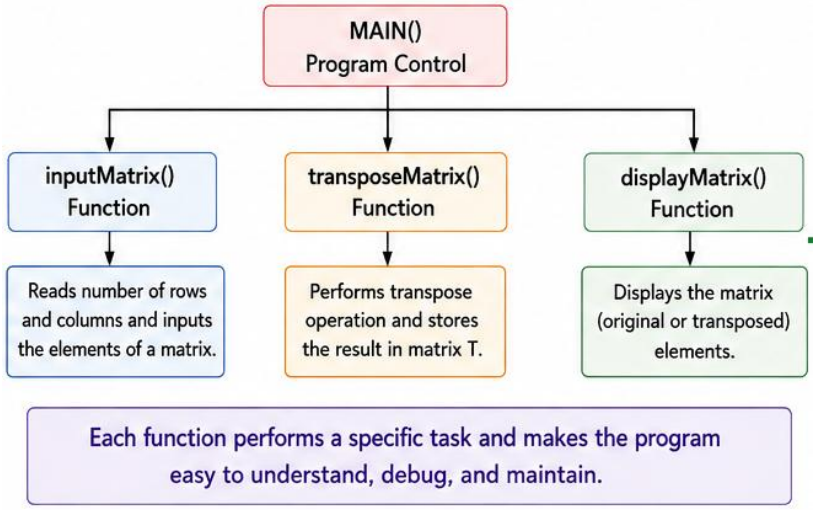


Figure. 4: Modular Programming Structure

ALOGRITHM:

1. Start.
2. Declare matrices A and T.
3. Enter the number of rows and columns of the matrix.
4. Call inputMatrix() to enter the elements of the matrix.
5. Call transposeMatrix() to find the transpose of the matrix.
6. In transposeMatrix(), interchange the row and column indices.
7. Store A[i][j] in T[j][i].
8. Call displayMatrix() to display the original matrix.
9. Call displayMatrix() again to display the transposed matrix.
10. Stop.

PROGRAM (C++):

```
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            cin >> matrix[i][j];
        }
    }
}

// Function to find transpose of matrix
void transposeMatrix(int A[][10], int T[][10], int rows, int cols)
{
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            T[j][i] = A[i][j];  // Interchange row and column
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
}

int main()
{
    int A[10][10], T[10][10];
    int rows, cols;

    cout << "Enter number of rows and columns of first matrix: ";
    cin >> rows >> cols;

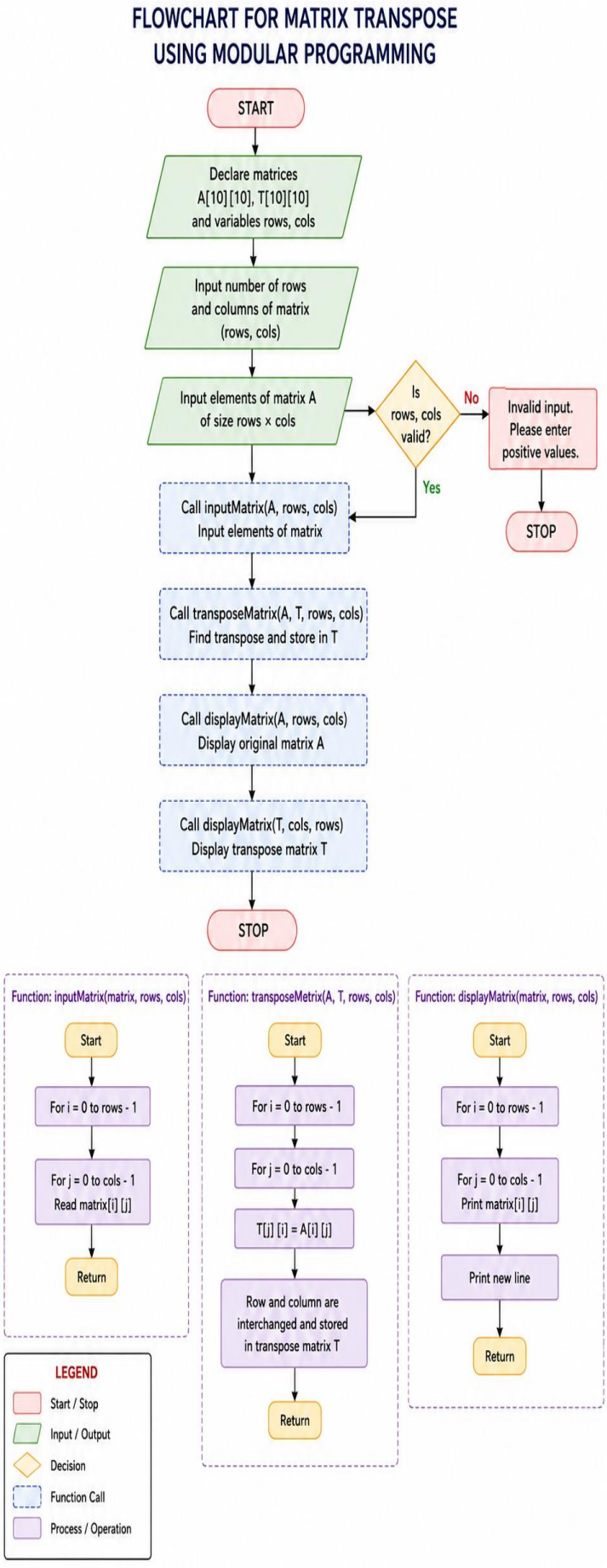
    cout << "Enter elements of matrix:\n";
    inputMatrix(A, rows, cols);

    transposeMatrix(A, T, rows, cols);

    cout << "\nOriginal Matrix:\n";
    displayMatrix(A, rows, cols);

    cout << "\nTranspose Matrix:\n";
    displayMatrix(T, cols, rows);

    return 0;
}
```



EXPLANATION OF THE PROGRAM:

Statement / Function	Explanation
int A[10][10]	Declares the original matrix.
int T[10][10]	Stores the transposed matrix.
rows, cols	Store the number of rows and columns of the original matrix.
inputMatrix()	Accepts the elements of the matrix from the user.
transposeMatrix()	Performs the transpose operation.
transpose[j][i] = matrix[i][j]	Interchanges the row and column positions of every element.
displayMatrix()	Displays the matrix elements row by row.
displayMatrix(A, rows, cols)	Displays the original matrix.
displayMatrix(T, cols, rows)	Displays the transpose matrix with rows and columns interchanged.
return 0;	Terminates the program successfully.

Sample Input 1 (Beginning):

Consider a matrix of order **2 × 3**.

Enter number of rows and columns of matrix: 2 3

Enter elements of matrix:

1 2 3

4 5 6

Calculation:

Original matrix:

A = [1 2 3]

[4 5 6]

Transpose operation:

T[0][0] = A[0][0] = 1

T[1][0] = A[0][1] = 2

T[2][0] = A[0][2] = 3

T[0][1] = A[1][0] = 4

T[1][1] = A[1][1] = 5

T[2][1] = A[1][2] = 6

Therefore,

T = [1 4]

[2 5]

[3 6]

Sample Output:

Original Matrix:

1 2 3

4 5 6

Transpose Matrix:

1 4

2 5

3 6

Time Complexity

For a matrix having m rows and n columns:

Time Complexity = O(m × n)

This is because the transpose operation visits every element of the matrix exactly once.

For a square matrix of size n × n:

Time Complexity = $O(n^2)$

Space Complexity

The program uses a separate matrix to store the transpose.

For a matrix of order m × n:

Space Complexity = O(m × n)

For a square matrix of size n × n:

Space Complexity = $O(n^2)$

IMPLEMENTATION OF CODE

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cin >> matrix[i][j];
        }
    }
}

// Function to find transpose of matrix
void transposeMatrix(int A[][10], int T[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            T[j][i] = A[i][j];
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
}

int main()
{
    int A[10][10], T[10][10];
    int rows, cols;

    // Input number of rows and columns
    cout << "Enter number of rows and columns of matrix: ";
    cin >> rows >> cols;

    // Input matrix elements
    cout << "Enter elements of matrix:" << endl;
    inputMatrix(A, rows, cols);

    // Find transpose
    transposeMatrix(A, T, rows, cols);

    // Display original matrix
    cout << "\nOriginal Matrix:" << endl;
    displayMatrix(A, rows, cols);

    // Display transpose matrix
    cout << "\nTranspose Matrix:" << endl;
    displayMatrix(T, cols, rows);

    return 0;
}
```

```
Enter number of rows and columns of matrix: 2 3
Enter elements of matrix:
1 2 3 4 5 6

Original Matrix:
1 2 3
4 5 6

Transpose Matrix:
1 4
2 5
3 6


...Program finished with exit code 0
Press ENTER to exit console.
```

APPLICATIONS: Matrix transpose is used in: Computer Graphics, Image Processing, Machine Learning, Artificial Intelligence, Scientific Computing, Engineering Calculations, Computer Vision, Mathematical and Numerical Computations, Transformation and Rotation Operations

RESULT: The program was successfully executed. It demonstrated **matrix transpose using modular programming** by using separate functions for **matrix input, matrix transpose, and matrix display**.

ASSIGNMENT:

- WAP to add two matrices using separate functions.
- WAP to subtract two matrices using modular programming.
- WAP to find the transpose of a matrix using a separate function.
- WAP to find the transpose of a 3 × 3 matrix using functions.
- WAP to check whether a given matrix is symmetric or not.